

Contents

TPV102 Dynamic Rupture Verification Guide	1
Table of Contents	2
1. Problem Description	2
2. Directory Layout	3
3. Architecture and Data Flow	4
3.1 Class Hierarchy	4
3.2 One-Step Data Flow	4
3.3 Parallel MPI Communication Points	5
4. Governing Equations	5
4.1 Velocity-Stress Wave Equation	5
4.2 DG Semi-Discretization	6
4.3 Godunov Upwind Flux	6
4.4 Boundary Conditions	9
4.5 Fault Riemann Solver Pipeline	9
4.6 State Evolution	14
4.7 PML Absorbing Layer	14
5. Code Walkthrough: Dynamic Module	15
5.1 WaveOperator<MeshType> (dynamic/wave_operator.hpp + .inl)	15
5.2 GodunovFlux (dynamic/godunov_flux.hpp/.cpp, 411 lines)	16
5.3 FaultFaceFlux (dynamic/fault_face_flux.hpp/.cpp, 159 lines)	16
5.4 FrictionSolver (dynamic/friction_solver.hpp/.cpp, 134 lines)	17
5.5 PMLayer (dynamic/pml_layer.hpp/.cpp, 95 lines)	17
5.6 SEASDynamicOperator<MeshType> (dynamic/seas_dynamic_operator.hpp)	17
6. Code Walkthrough: TPV102 Setup	17
6.1 TPV102Params (config/tpv102_params.hpp, 144 lines)	17
6.2 tpv102_setup.hpp (dynamic/tpv102_setup.hpp, 469 lines)	18
7. Driver Pipeline	18
8. Mesh Generation	19
9. Benchmark Data and Verification	20
9.1 Reference Solutions	20
9.2 Station Format (SCEC Standard)	20
9.3 SCEC Fault Stations (9 locations)	20
9.4 Verification Targets	21
10. Testing	21
Unit Tests (make test-tpv102-setup)	21
Integration Tests (make test-tpv102-local)	21
11. Running on HPC	21
Build	21
Run Locally (coarse mesh)	21
Run on Frontera (production)	22
CLI Arguments	22

TPV102 Dynamic Rupture Verification Guide

Codebase roadmap for the SCEC TPV102 dynamic rupture benchmark implementation. Covers the physics, code architecture, file layout, execution flow, and verification strategy. Written for someone new to this code who needs to understand, modify, or extend the dynamic rupture module.

Last updated: 2026-04-14 (v3: fault face dispatch in WaveOperator, state evolution via UpdateStateAnalytic, updated line counts and mesh inventory).

For a rendered version:

pandoc TPV102_GUIDE.md -o TPV102_GUIDE.html --standalone --embed-resources --mathjax --toc
open TPV102_GUIDE.html

Table of Contents

1. Problem Description
2. Directory Layout
3. Architecture and Data Flow
4. Governing Equations
5. Code Walkthrough: Dynamic Module
6. Code Walkthrough: TPV102 Setup
7. Driver Pipeline
8. Mesh Generation
9. Benchmark Data and Verification
10. Testing
11. Running on HPC

1. Problem Description

SCEC TPV102 is a 3D dynamic rupture benchmark problem. A vertical strike-slip fault embedded in an elastic half-space is nucleated by a time-dependent shear stress perturbation. The rupture propagates bilaterally along-strike and is arrested by velocity-strengthening friction at the fault edges. Rate-and-state friction with the aging law governs fault slip.

Key differences from the quasi-dynamic SEAS benchmarks (BP1/BP2/BP5):

Property	SEAS (BP5)	Dynamic (TPV102)
PDE	$\nabla \cdot \sigma = \mathbf{0}$ (quasi-static)	$\rho \ddot{\mathbf{u}} = \nabla \cdot \sigma$ (wave equation)
Formulation	Implicit: $K\mathbf{u} = \mathbf{b}$ at every step	Explicit: $\dot{Q} = M^{-1}F(Q)$ (no global solve)
Time scale	$\sim 10^3$ years	~ 12 seconds
Time stepper	Adaptive RK45 (PETSc TS)	Fixed-dt RK4 (CFL-limited)
DG method	IP/BR2 (second-order form)	Godunov upwind (first-order velocity-stress)
Boundaries	Dirichlet (plate rate)	Absorbing (first-order ABC or PML)
Coupling	Slip $\rightarrow$ traction via global solve	Fault Riemann solver per face (element-local)

Physical parameters (from config/tpv102_params.hpp):

Parameter	Symbol	Value	Unit
Density	ρ	2670	kg/m ³
S-wave speed	c_s	3464	m/s
P-wave speed	c_p	6000	m/s
Shear modulus	$\mu = \rho c_s^2$	3.203×10^{10}	Pa
Normal stress	σ_n	120	MPa
Initial shear stress	τ_{ini}	75	MPa
Reference friction	f_0	0.6	—
State evolution	b	0.012	—
Critical slip distance	D_c	0.02	m

Parameter	Symbol	Value	Unit
VW direct effect	a_{vw}	0.008	–
VS direct effect	a_{vs}	0.016	–
Nucleation stress	$\Delta\tau$	25	MPa
Nucleation radius	R	3	km
Nucleation rise time	T	1.0	s
Fault extent		30 km $\times$ 15 km	
Domain		60 km $\times$ 60 km $\times$ 30 km	

2. Directory Layout

tpv102/	# Benchmark-specific data
mesh/	
tpv102_200m.geo	# Gmsh: h=200m (development/convergence)
tpv102_100m.geo	# Gmsh: h=100m (production)
benchmark_data/	
scec_drdg3d/	# DR-DG3D reference (9 stations)
scec_pylith/	# PyLith reference (9 stations)
benchmark_document/	# SCEC spec PDFs
scripts/	
frontera_tpv102.sbatch	# HPC job script (4 nodes, 224 tasks)
dynamic/	# Dynamic rupture module (shared)
wave_state.hpp	# State vector $Q = [\text{sigma}, v]$, 9 components
wave_operator.hpp	# Template WaveOperator<MeshType> declaration (159 lines)
wave_operator.inl	# Template implementation (717 lines)
wave_operator.cpp	# Stub (template impl in .inl)
godunov_flux.hpp / .cpp	# Godunov upwind flux (411 lines impl)
fault_face_flux.hpp / .cpp	# Fault Riemann solver (159 lines impl)
friction_solver.hpp / .cpp	# Brent/Newton friction solver (134 lines impl)
pml_layer.hpp / .cpp	# PML absorbing boundaries (95 lines impl)
seas_dynamic_operator.hpp	# Template coupling wrapper
tpv102_setup.hpp	# TPV102 init, stations, nucleation (469 lines)
config/	
tpv102_params.hpp	# Physical parameters + spatial functions
drivers/	
tpv102_driver.cpp	# Parallel MPI driver (397 lines)
tests/	
unit/test_tpv102_setup.cpp	# Parameter unit tests (190 lines)
verification/test_tpv102_local.cpp	# Integration tests L1-L6 (547 lines)

Total dynamic module size: ~4,200 lines of C++ (excluding comments, mesh files, docs).

3. Architecture and Data Flow

3.1 Class Hierarchy

Both `WaveOperator` and `SEASDynamicOperator` are **templates on `MeshType`** (`Mesh` for serial, `ParMesh` for parallel). This allows the same code to handle both cases:

```
TimeDependentOperator
|
|  +-- WaveOperator<MeshType>          # DG semi-discrete wave equation
|  |   owns: GodunovFlux              # Upwind numerical flux
|  |   owns: FaultBasis               # Coordinate frames on fault faces
|  |   uses: PMLayer*                # Optional absorbing layer (non-owning)
|  |
|  +-- SEASDynamicOperator<MeshType>  # Coupling wrapper
|  |   uses: WaveOperator<MeshType>*  # Non-owning
|  |   uses: FaultFaceFlux*          # Non-owning (fault Riemann solver)
|  |   owns: FrictionSolver           # Brent/Newton for friction equation
```

3.2 One-Step Data Flow

Each RK4 stage calls `WaveOperator::Mult(Q, dQdt)`, which performs:

```
WaveOperator<MeshType>::Mult(Q, dQdt)          [wave_operator.inl:100]
|
|  Step 1: Volume integral (element-local, no communication)
|  ComputeVolumeRHS(Q, dQdt)                   [wave_operator.inl:129]
|  For each element e, for each quad point q:
|  Compute flux F_j[c] = A_j[c,d] * Q_e[d]      (A_j = Jacobian matrix)
|  Accumulate: dQdt[c] += w * dshape(i,j) * F_j[c]
|
|  Step 2: Face flux (Godunov upwind, element-pair communication)
|  ComputeFaceFluxRHS(Q, dQdt)                 [wave_operator.inl:198]
|  For each interior face:
|  GodunovFlux::Interior(nor, Q_self, Q_nbr, F_h)
|  dQdt_elem1 -= w * shape(i) * F_h[c]          (subtract from self)
|  dQdt_elem2 += w * shape(i) * F_h[c]          (add to neighbor)
|  For each absorbing boundary face (attr 5):
|  GodunovFlux::Absorbing(nor, Q_self, F_h)
|  For each free surface face (attr 1):
|  GodunovFlux::FreeSurface(nor, Q_self, F_h)
|  For each fault face (attr 3):                 [wave_operator.inl:306-423]
|  Rotate Q± to fault-local via BuildRotation/BuildRotationInverse
|  FaultFaceFlux::Evaluate(...)                 (trial -> friction -> imposed)
|  Rotate imposed states back to global
|  GodunovFlux::Interior(nor, Q_imp+, Q_imp-)   (standard accumulation)
|
|  Step 2b: Shared face flux (parallel only)
|  ComputeSharedFaceFluxRHS(Q, dQdt)           [wave_operator.inl:449]
|  Per-component ghost Q exchange via ParGridFunction::ExchangeFaceNbrData()
|  Godunov flux with actual ghost data; accumulate into local element only
|
|  Step 3: PML damping (if enabled)
|  ApplyPMLDamping(Q, dQdt)                   [wave_operator.inl:649]
|  dQdt -= d(x) * D * Q   for DOFs inside PML region
```

```
| Step 4: Inverse mass matrix (element-local, no communication)
|   ApplyMassInverse(dQdt)                                [wave_operator.inl:573]
|   dQdt[e] = M_e^{-1} * dQdt[e]    (dense matvec per element)
```

3.3 Parallel MPI Communication Points

RK4 time loop (tpv102_driver.cpp:294)

```
|
|   wave.Mult(Q, k_stage):
|     +-- ComputeSharedFaceFluxRHS()          -- MPI: face neighbor data exchange
|         Per-component ParGridFunction exchange (R-001 fix)
|         Face neighbor structures initialized once in constructor (R-005 fix)
|
|   After each accepted step:
|     +-- UpdateStateAnalytic()                -- R-003/R-004: aging law + slip accumulation
|     +-- MPI_Allreduce(V_max_local, MAX)      -- R-006: consistent V_max across ranks
|     +-- MPI_Allreduce(local_nan, MAX)        -- R-003: NaN detection across ranks
|
|   Station output:
|     +-- TPV102StationWriter::Open()          -- R-004: global distance resolution
|         MPI_Allreduce(local_dist, MIN) to determine which rank owns each station
```

4. Governing Equations

4.1 Velocity-Stress Wave Equation

The first-order velocity-stress formulation of 3D linear elasticity:

$$\frac{\partial Q}{\partial t} + A_x \frac{\partial Q}{\partial x} + A_y \frac{\partial Q}{\partial y} + A_z \frac{\partial Q}{\partial z} = 0$$

where the state vector Q has 9 components:

$$Q = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z]^T$$

State vector indexing (wave_state.hpp, line 28):

```
enum QIndex : int {
    SXX = 0,  SYY = 1,  SZZ = 2,    // normal stresses
    SXY = 3,  SYZ = 4,  SXZ = 5,    // shear stresses
    VX  = 6,  VY  = 7,  VZ  = 8,    // particle velocities
};
```

The Jacobian matrices A_x, A_y, A_z are constant 9×9 matrices determined by λ, μ, ρ . They encode the constitutive law ($\dot{\sigma} = \mathbb{C} : \dot{\varepsilon}$) and momentum balance ($\rho \dot{v} = \nabla \cdot \sigma$).

A_x matrix structure (nonzero entries only):

Row	Col	Value	Physics
σ_{xx}	v_x	$-(\lambda + 2\mu)$	P-wave coupling
σ_{yy}	v_x	$-\lambda$	Poisson effect
σ_{zz}	v_x	$-\lambda$	Poisson effect
σ_{xy}	v_y	$-\mu$	S-wave (y -polarized)
σ_{xz}	v_z	$-\mu$	S-wave (z -polarized)

Row	Col	Value	Physics
v_x	σ_{xx}	$-1/\rho$	x -momentum
v_y	σ_{xy}	$-1/\rho$	y -momentum
v_z	σ_{xz}	$-1/\rho$	z -momentum

A_y and A_z follow by cyclic permutation of indices. See `BuildJacobian()` in `godunov_flux.cpp:125-188` for the general construction.

4.2 DG Semi-Discretization

Multiply by test function ϕ_i , integrate over element K , integrate by parts:

$$M_K \frac{dQ_K}{dt} = \underbrace{\int_K \nabla \phi_i \cdot (A_x Q, A_y Q, A_z Q) dx}_{\text{volume integral}} - \underbrace{\oint_{\partial K} \phi_i \hat{F} \cdot \mathbf{n} ds}_{\text{face flux}}$$

where M_K is the element mass matrix and $\hat{F}$ is the numerical flux.

4.3 Godunov Upwind Flux

4.3.1 Why an upwind flux is needed The DG semi-discretization (Section 4.2) produces a face integral $\oint_{\partial K} \phi_i \hat{F} \cdot \mathbf{n} ds$. At each face, Q is double-valued (one value from each element). The numerical flux $\hat{F}$ selects a single value by solving the Riemann problem at the face. For the linear hyperbolic system $\partial Q / \partial t + A_n \partial Q / \partial n = 0$, the exact Riemann solution is the Godunov flux (upwind splitting).

4.3.2 The Jacobian matrix A_x The x -direction Jacobian encodes both the constitutive law and momentum balance. For isotropic elasticity with Lamé parameters λ , μ and density ρ :

Stress rows (from $\dot{\sigma}_{ij} = C_{ijkl} \dot{\varepsilon}_{kl}$, keeping only $\partial/\partial x$ terms):

$$A[\sigma_{ij}, v_k] = -C_{ij,k,x}$$

where the isotropic stiffness $C_{ij,k,\ell} = \lambda \delta_{ij} \delta_{k\ell} + \mu(\delta_{ik} \delta_{j\ell} + \delta_{i\ell} \delta_{jk})$.

Velocity rows (from $\rho \dot{v}_k = \partial \sigma_{kx} / \partial x$):

$$A[v_k, \sigma_{kx}] = -1/\rho$$

Code (`godunov_flux.cpp:125-188`): `BuildJacobian(dir, A)` implements this for any direction `dir` $\in \{0, 1, 2\}$ using a Voigt index helper. Each nonzero entry is assembled from the triple-loop over (i, j, k) .

4.3.3 Eigenvalue decomposition of A_x The 9×9 Jacobian A_x has **nine eigenvalues**:

$$\lambda_1 = +c_p, \quad \lambda_{2,3} = +c_s, \quad \lambda_{4,5,6} = 0, \quad \lambda_{7,8} = -c_s, \quad \lambda_9 = -c_p$$

where $c_p = \sqrt{(\lambda + 2\mu)/\rho}$ (P-wave speed) and $c_s = \sqrt{\mu/\rho}$ (S-wave speed).

The eigenvalues have a physical meaning: $- +c_p$: rightgoing P-wave (compressional) - $+c_s$: rightgoing S-waves (shear, two polarizations) - 0 : three zero-speed modes ($\sigma_{yy}, \sigma_{zz}, \sigma_{yz}$ — stresses with no x indices) - $-c_s, -c_p$: leftgoing S and P waves

The corresponding right eigenvectors (columns of R) describe how each wave mode couples stress and velocity:

Code (`godunov_flux.cpp:57-83`): The eigenvector matrix R is built analytically. For example, column 0 ($\lambda = +c_p$) sets $R(SXX, 0) = -(\text{lambda} + 2*\mu)$, $R(SYY, 0) = -\text{lambda}$, $R(SZZ, 0) = -\text{lambda}$, $R(VX, 0) = \text{cp_}$. This encodes the P-wave: compression in σ_{xx} drives velocity v_x , while σ_{yy} and σ_{zz} follow via Poisson coupling.

4.3.4 Split flux matrices A_x^+ and A_x^- Given $A_x = R\Lambda R^{-1}$, define:

$$A_x^+ = R\Lambda^+R^{-1}, \quad A_x^- = R\Lambda^-R^{-1}$$

where $\Lambda^+ = \text{diag}(\max(\lambda_i, 0))$ and $\Lambda^- = \text{diag}(\min(\lambda_i, 0))$.

A_x^+ propagates rightgoing waves (the information leaving the self element), A_x^- propagates leftgoing waves (the information arriving from the neighbor). By construction $A_x = A_x^+ + A_x^-$.

Code (`godunov_flux.cpp:91-119`): The diagonal matrices `Lambda_plus` and `Lambda_minus` are filled, then multiplied: `Ax_plus_ = R * Lambda_plus * R^{-1}`. These are precomputed once in the constructor and reused for every face.

4.3.5 Interior flux formula For a face with unit outward normal $\mathbf{n}$ (from Elem1 to Elem2), the normal Jacobian is:

$$A_n = n_x A_x + n_y A_y + n_z A_z$$

The Godunov flux splits A_n by eigenvalue sign:

$$\hat{F} = A_n^+ Q_{\text{self}} + A_n^- Q_{\text{neighbor}}$$

How this ensures a unique flux (resolves the double-valued Q). At every face, the DG solution has two values: Q_L (from the left element) and Q_R (from the right). The 1D Riemann problem in the normal direction asks: if Q_L and Q_R are separated by a discontinuity at $x = 0$, what is the exact solution at $x = 0$ as $t \rightarrow 0^+$?

For a linear system $\partial Q / \partial t + A_n \partial Q / \partial x = 0$, the exact solution decomposes Q_L and Q_R into eigenmodes. Each mode travels at speed λ_i . The state at $x = 0$ is:

- From the left (Q_L): keep only the rightgoing waves ($\lambda_i > 0$), i.e., $A_n^+ Q_L$
- From the right (Q_R): keep only the leftgoing waves ($\lambda_i < 0$), i.e., $A_n^- Q_R$

The flux $\hat{F} = A_n^+ Q_L + A_n^- Q_R$ is the physical flux $A_n Q^*$ evaluated at the unique Riemann solution Q^* . Equivalently:

$$\hat{F} = \frac{1}{2}(A_n + |A_n|)Q_L + \frac{1}{2}(A_n - |A_n|)Q_R = \frac{1}{2}A_n(Q_L + Q_R) - \frac{1}{2}|A_n|(Q_R - Q_L)$$

The first term is the centered flux; the second is a diffusion-like upwind correction proportional to the jump $Q_R - Q_L$, scaled by the absolute eigenvalues. For smooth solutions ($Q_L \approx Q_R$), the correction vanishes and $\hat{F} \approx A_n Q$.

4.3.6 Implementation via rotation Computing A_n^+ and A_n^- for every face normal would be expensive. Instead, the code rotates Q to a face-local frame where $\mathbf{n}$ becomes the x -axis, applies the precomputed A_x^+ and A_x^- , and rotates back. The rotation is a 9×9 block-diagonal matrix acting on both stress (Voigt) and velocity blocks.

Step-by-step (`godunov_flux.cpp`, `Interior()` at line 325):

```

1. BuildFrame(nor, t1, t2)          -- Gram-Schmidt: pick t1 perp nor, t2 = nor x t1
2. BuildRotationInverse(nor,...,Tinv) -- 9x9 matrix: Q_local = T^{-1} * Q_global
3. Q_self_rot = T^{-1} * Q_self      -- rotate self state to face-local
4. Q_nbr_rot  = T^{-1} * Q_nbr       -- rotate neighbor state
5. F_rot = Ax_plus * Q_self_rot      -- outgoing waves from self (precomputed)
      + Ax_minus * Q_nbr_rot        -- incoming waves from neighbor
6. F_h = T * F_rot                  -- rotate flux back to global frame

```

Why this works: For any orthogonal frame $(\mathbf{n}, \mathbf{t}_1, \mathbf{t}_2)$, $A_n = T A_x T^{-1}$, so $A_n^+ Q = T A_x^+ T^{-1} Q$ and the split commutes with rotation.

Rotation details (godunov_flux.cpp:196-276): - Velocity block: standard 3×3 rotation $v_{\text{local}} = Q v_{\text{global}}$ where Q has rows $(\mathbf{n}, \mathbf{t}_1, \mathbf{t}_2)$. - Stress block: Voigt tensor rotation $\sigma'_{ab} = Q_{ai} Q_{bj} \sigma_{ij}$ with symmetry factors for off-diagonal Voigt entries.

4.3.7 Absorbing and free-surface boundary fluxes **Absorbing** ($\hat{F} = A_n^+ Q_{\text{self}}$): Sets $Q_{\text{nbr}} = 0$ (no incoming waves), which means A_n^- contributes nothing — only outgoing waves leave the domain.

Code (godunov_flux.cpp:354-361): `Absorbing()` calls `Interior(nor, Q_self, Q_zero, F_h)`.

Free surface ($\sigma \cdot \mathbf{n} = 0$): We need to construct a ghost state Q_{ghost} outside the domain such that the Godunov flux $\hat{F} = A_n^+ Q_{\text{self}} + A_n^- Q_{\text{ghost}}$ enforces zero normal traction at the boundary. The construction works in the face-normal rotated frame.

The mirror matrix Γ . In the rotated frame ($x = \text{normal}$, $y = \text{tangent1}$, $z = \text{tangent2}$), the 9-component state vector has stress and velocity entries. We classify each stress component by how many normal (x) indices it has:

Component	Rotated meaning	Normal indices	Γ	Reason
SXX (0)	σ_{nn}	2 (even)	-1	Must vanish at interface
SYX (1)	$\sigma_{t_1 t_1}$	0	+1	No constraint
SZZ (2)	$\sigma_{t_2 t_2}$	0	+1	No constraint
SXY (3)	σ_{nt_1} (shear)	1 (odd)	-1	Must vanish at interface
SYZ (4)	$\sigma_{t_1 t_2}$	0	+1	No constraint
SXZ (5)	σ_{nt_2} (shear)	1 (odd)	-1	Must vanish at interface
VX (6)	v_n	-	+1	Unconstrained
VY (7)	v_{t_1}	-	+1	Unconstrained
VZ (8)	v_{t_2}	-	+1	Unconstrained

So $\Gamma = \text{diag}(-1, +1, +1, -1, +1, -1, +1, +1, +1)$ and the ghost state is $Q_{\text{ghost}} = \Gamma Q_{\text{self}}$.

Why this works (proof). The Godunov flux at the boundary is:

$$\hat{F} = A_x^+ Q_{\text{self}} + A_x^- Q_{\text{ghost}} = A_x^+ Q + A_x^- \Gamma Q$$

The Riemann solution at an interface gives the interface state Q^* (the state “seen” at $x = 0$). For a linear system, this interface state is:

$$Q^* = (A_x^+)^{-1} A_x^+ Q + (A_x^-)^{-1} A_x^- \Gamma Q$$

But more directly: with $Q_{\text{ghost}} = \Gamma Q$, the average traction at the interface (from the characteristic relations on each side) gives, for the normal stress:

$$\sigma_{nn}^* = \frac{Z_p \sigma_{nn}^{\text{self}} + Z_p \sigma_{nn}^{\text{ghost}} + Z_p^2 (v_n^{\text{ghost}} - v_n^{\text{self}})}{2Z_p}$$

Since $\sigma_{nn}^{\text{ghost}} = -\sigma_{nn}^{\text{self}}$ and $v_n^{\text{ghost}} = v_n^{\text{self}}$:

$$\sigma_{nn}^* = \frac{Z_p \sigma_{nn} - Z_p \sigma_{nn} + 0}{2Z_p} = 0 \quad \checkmark$$

Similarly for the shear tractions σ_{nt_k} :

$$\tau_k^* = \frac{Z_s \tau_k + Z_s (-\tau_k) + Z_s^2 (v_{t_k} - v_{t_k})}{2Z_s} = 0 \quad \checkmark$$

The traction components that couple to the normal direction (σ_{nn} , σ_{nt_1} , σ_{nt_2}) all vanish at the interface, which is exactly $\sigma \cdot \mathbf{n} = 0$. The tangential stresses ($\sigma_{t_1 t_1}$, $\sigma_{t_2 t_2}$, $\sigma_{t_1 t_2}$) are unconstrained and propagate freely. Velocities are also unconstrained — the free surface can move.

Code (godunov_flux.cpp:366-408): `FreeSurface()` rotates to face-normal frame, applies `gamma[]` to create the ghost, then calls `ApplySplitFlux(Q_rot, Q_ghost_rot, F_rot)` and rotates back.

4.4 Boundary Conditions

BC Type	Mesh Attr	Formula	Physical Meaning
Absorbing	5	$\hat{F} = A_n^+ Q_{\text{self}}$	Outgoing waves only, no reflection
Free surface	1	Mirror stress via Γ matrix	$\sigma \cdot \mathbf{n} = 0$
Fault	3	Fault Riemann solver (Section 4.5)	Friction-constrained interface

4.5 Fault Riemann Solver Pipeline

On fault faces, the standard Godunov flux is replaced by a friction-constrained Riemann solver. Instead of finding a single state at the interface (as the Godunov flux does), we find two imposed states $Q^{+, \text{imp}}$ and $Q^{-, \text{imp}}$ — one for each element — that simultaneously satisfy the friction law and the elastic wave characteristic relations. The pipeline has five steps.

All equations below are in **fault-local coordinates**: x = fault normal, y = tangent 1, z = tangent 2. In code, the rotation to/from these coordinates is handled by `BuildRotation` / `BuildRotationInverse` in `wave_operator.inl:327-342`.

Step 1: Trial traction (Eq. 7) — what would happen if the fault were locked

Notation. The trial quantities $\Delta\sigma_n$, $\Delta\tau_1$, $\Delta\tau_2$ are *perturbations* from the background stress ($\sigma_{n,0}$, $\tau_{1,0}$, $\tau_{2,0}$) stored in `DOFData`. The code variable names `sigma_n_trial`, `tau1_trial`, `tau2_trial` correspond to these increments.

Physical idea. Imagine the fault is welded shut ($V = 0$). Waves arriving from both sides create a stress state at the interface. This “locked-fault” stress is the *trial traction* — the elastic demand on the fault.

Derivation from characteristic relations. Consider a 1D Riemann problem in the fault-normal direction. The characteristic relation for a P-wave arriving at the interface from the $+$ side (traveling in the $-n$ direction) is:

$$\sigma_n^* - \sigma_n^+ = -Z_p^+ (v_n^* - v_n^+) \quad (\text{C1})$$

From the $-$ side (traveling in the $+n$ direction):

$$\sigma_n^* - \sigma_n^- = +Z_p^- (v_n^* - v_n^-) \quad (\text{C2})$$

These are the Rankine-Hugoniot jump conditions for each wavefront. $Z_p^\pm = \rho^\pm c_p^\pm$ is the P-wave impedance on each side, and $*$ denotes the interface state.

Locked-fault constraint. If the fault is locked: $v_n^{*,+} = v_n^{*,-} = v_n^*$ (no velocity discontinuity). From (C1): $v_n^* = v_n^+ - (\sigma_n^* - \sigma_n^+)/Z_p^+$. From (C2): $v_n^* = v_n^- + (\sigma_n^* - \sigma_n^-)/Z_p^-$. Equating:

$$v_n^+ - \frac{\sigma_n^* - \sigma_n^+}{Z_p^+} = v_n^- + \frac{\sigma_n^* - \sigma_n^-}{Z_p^-}$$

$$\sigma_n^* \left(\frac{1}{Z_p^+} + \frac{1}{Z_p^-} \right) = v_n^- - v_n^+ + \frac{\sigma_n^+}{Z_p^+} + \frac{\sigma_n^-}{Z_p^-}$$

Defining $\eta_p = Z_p^+ Z_p^- / (Z_p^+ + Z_p^-)$ (harmonic mean impedance):

$$\Delta\sigma_n = \eta_p \left(v_n^- - v_n^+ + \frac{\sigma_n^+}{Z_p^+} + \frac{\sigma_n^-}{Z_p^-} \right) \quad (7a)$$

The same derivation for S-waves (shear, two polarization components) gives:

$$\Delta\tau_1 = \eta_s \left(v_{t_1}^- - v_{t_1}^+ + \frac{\tau_1^+}{Z_s^+} + \frac{\tau_1^-}{Z_s^-} \right) \quad (7b)$$

$$\Delta\tau_2 = \eta_s \left(v_{t_2}^- - v_{t_2}^+ + \frac{\tau_2^+}{Z_s^+} + \frac{\tau_2^-}{Z_s^-} \right) \quad (7c)$$

where $\eta_s = Z_s^+ Z_s^- / (Z_s^+ + Z_s^-)$ and $Z_s = \rho c_s$. For homogeneous material: $\eta_s = Z_s/2$.

Code (`fault_face_flux.cpp:37-63`): `ComputeTrialTraction()` directly implements Eqs. (7a-c). In fault-local coordinates, $\text{SXX} = \sigma_n$, $\text{SXY} = \tau_1$, $\text{SXZ} = \tau_2$, $\text{VX} = v_n$, $\text{VY} = v_{t_1}$, $\text{VZ} = v_{t_2}$. The code names `sigma_n_trial`, `tau1_trial`, `tau2_trial` are the increments $\Delta\sigma_n$, $\Delta\tau_1$, $\Delta\tau_2$.

Step 2: Friction solve (Eq. 8) and why $\eta_s \hat{V}$ is exact The total traction combines the background stress (stored in `DOFData`) with the trial increment:

$$\sigma_n^{\text{total}} = \sigma_{n,0} + \Delta\sigma_n, \quad \Theta = \sqrt{(\tau_{1,0} + \Delta\tau_1)^2 + (\tau_{2,0} + \Delta\tau_2)^2}$$

Code (`fault_face_flux.cpp:79-84`): Lines 79-81 compute the total traction; line 84 computes Θ .

Rigorous proof that slip reduces traction by exactly $\eta_s \hat{V}$.

When the fault slips, the two sides have different tangential velocities: $v_t^{*,+} - v_t^{*,-} = \hat{V}$ (the slip rate). We now re-derive the interface traction allowing this discontinuity.

The characteristic relations for S-waves (one component) are:

$$\tau^* - \tau^+ = -Z_s^+ (v_t^{*,+} - v_t^+) \quad (\text{S1: from + side})$$

$$\tau^* - \tau^- = +Z_s^- (v_t^{*,-} - v_t^-) \quad (\text{S2: from - side})$$

Note that both sides see the *same* corrected traction τ^* (Newton's third law: the fault face has no mass, so traction is continuous across it).

$$\text{From (S1): } v_t^{*,+} = v_t^+ - (\tau^* - \tau^+)/Z_s^+$$

$$\text{From (S2): } v_t^{*,-} = v_t^- + (\tau^* - \tau^-)/Z_s^-$$

The slip rate is:

$$\hat{V} = v_t^{*,+} - v_t^{*,-} = \underbrace{(v_t^+ - v_t^-)}_{\text{existing velocity jump}} - \underbrace{\frac{\tau^* - \tau^+}{Z_s^+} - \frac{\tau^* - \tau^-}{Z_s^-}}_{\text{traction-induced velocity change}}$$

Expanding the second part:

$$= (v_t^+ - v_t^-) - \tau^* \left(\frac{1}{Z_s^+} + \frac{1}{Z_s^-} \right) + \frac{\tau^+}{Z_s^+} + \frac{\tau^-}{Z_s^-}$$

Recall the locked-fault solution (Eq. 7): $\Delta\tau_{\text{locked}}/\eta_s = v_t^- - v_t^+ + \tau^+/Z_s^+ + \tau^-/Z_s^-$, and $1/\eta_s = 1/Z_s^+ + 1/Z_s^-$. So:

$$\hat{V} = -(v_t^- - v_t^+) - \frac{\tau^*}{\eta_s} + \frac{\tau^+}{Z_s^+} + \frac{\tau^-}{Z_s^-} = -\frac{\Delta\tau_{\text{locked}}}{\eta_s} + \frac{v_t^- - v_t^+}{\cancel{\lambda}} \cdot \cancel{\lambda}$$

More directly: rearranging for τ^* :

$$\frac{\tau^*}{\eta_s} = \underbrace{v_t^- - v_t^+ + \frac{\tau^+}{Z_s^+} + \frac{\tau^-}{Z_s^-}}_{=\Delta\tau_{\text{locked}}/\eta_s \text{ from (7b)}} - \hat{V}$$

$$\boxed{\tau^* = \Delta\tau_{\text{locked}} - \eta_s \hat{V}}$$

This is **exact** — no Taylor expansion, no small-parameter approximation. The $\eta_s \hat{V}$ term arises directly from the characteristic compatibility relations of the two-sided Riemann problem. Physically, η_s is the radiation

impedance: each unit of slip rate radiates shear waves into both half-spaces, reducing the interface traction by exactly η_s per unit slip rate.

The friction balance equation. The friction law requires that the actual traction magnitude equals the friction strength: $|\tau_{\text{total}}^*| = |\sigma_n^{\text{total}}| \cdot f(\hat{V}, \psi)$. Using the relation above for the total traction:

$$\Theta - \eta_s \hat{V} = |\sigma_n^{\text{total}}| \cdot f(\hat{V}, \psi)$$

Rearranging:

$$|\sigma_n^{\text{total}}| \cdot f(\hat{V}, \psi) + \eta_s \hat{V} = \Theta \quad (8)$$

where the regularized friction coefficient is:

$$f(V, \psi) = a \cdot \text{asinh} \left[\frac{V}{2V_0} \exp(\psi/a) \right]$$

The left-hand side is the friction resistance ($|\sigma_n|f$) plus the exact radiation damping ($\eta_s \hat{V}$); the right-hand side is the elastic demand (Θ).

Note. For homogeneous material, $\eta_s = Z_s/2 = \rho c_s/2$. In the quasi-dynamic (QD) approximation, the *same* $\eta_s V$ term appears, but it is derived differently (as a radiation damping approximation). In the dynamic formulation, $\eta_s V$ is exact — it is the Riemann solution, not an approximation. The mathematical coincidence that QD and dynamic use the same η_s is why the same `SolveSlipRatePsi()` solver works for both.

Bracket for Brent. Define $g(V) = |\sigma_n|f(V, \psi) + \eta_s V - \Theta$. At $V = 0$: $g(0) = -\Theta < 0$. At $V = \Theta/\eta_s$: $g = |\sigma_n|f(\dots) > 0$. So the root is always bracketed in $[0, \Theta/\eta_s]$.

Code (`friction_solver.cpp:28-35`): `Residual()` evaluates $g(V)$. `SolveBrent()` (line 70-75) delegates to the proven QD solver `DieterichRuinaFriction::SolveSlipRatePsi` which works in $\log_{10}(V)$ space.

Step 3: Decompose slip rate into components (Eq. 9) The scalar $\hat{V}$ is the slip rate magnitude. The direction must align with the total traction vector (the fault slips in the direction the stress pushes it):

$$\text{strength} = |\sigma_n^{\text{total}}| \cdot f(\hat{V}, \psi)$$

$$V_1 = \hat{V} \frac{\tau_{1,0} + \Delta\tau_1}{\text{strength} + \eta_s \hat{V}}, \quad V_2 = \hat{V} \frac{\tau_{2,0} + \Delta\tau_2}{\text{strength} + \eta_s \hat{V}} \quad (9)$$

Note: the denominator $\text{strength} + \eta_s \hat{V} = \Theta$ (from Eq. 8), so this is equivalent to $V_k = \hat{V} \cdot \tau_{k,\text{total}}/\Theta$.

Code (`fault_face_flux.cpp:100-107`): Lines 101-103 compute friction strength, lines 106-107 decompose slip rate.

Step 4: Corrected traction (Eq. 10) The actual traction increment on the fault is the trial increment reduced by the exact radiation damping $\eta_s V_k$:

$$\Delta\tau_1^{\text{corr}} = \Delta\tau_1 - \eta_s V_1, \quad \Delta\tau_2^{\text{corr}} = \Delta\tau_2 - \eta_s V_2 \quad (10)$$

Normal traction is unchanged by tangential slip: $\Delta\sigma_n^{\text{corr}} = \Delta\sigma_n$.

The *total* corrected traction reported to the output is $\tau_{k,0} + \Delta\tau_k^{\text{corr}}$.

Code (fault_face_flux.cpp:110-114): The code variable `tau1_corr` stores $\Delta\tau_1^{\text{corr}}$ (the increment). Lines 153-155 add back the background to store the total corrected traction in `data.tau1_corr`.

Step 5: Imposed states (Eq. 11-12) We now know the corrected traction increments $\Delta\sigma_n^{\text{corr}}$, $\Delta\tau_k^{\text{corr}}$. We need to construct $Q^{\pm, \text{imp}}$ — the states that each element “sees” at the fault face — so that:

1. **Traction continuity:** both sides see the corrected traction.
2. **Characteristic consistency:** no spurious wave reflections.
3. **Correct slip:** $v_t^{+, \text{imp}} - v_t^{-, \text{imp}} = V_k$.

The imposed states are derived from the same characteristic relations (S1), (S2) used in Step 2. Now that we know τ^* (the corrected traction), we solve for the velocities on each side. From (S2): $v_t^{*, -} = v_t^- + (\tau^* - \tau^-)/Z_s^-$. From (S1): $v_t^{*, +} = v_t^+ - (\tau^* - \tau^+)/Z_s^+$.

Since Q is a perturbation field (background stress lives in `DOFData`), the stress components in $Q^{\pm}$ are increments. The corrected traction in the imposed state is $\Delta\tau_k^{\text{corr}}$ and $\Delta\sigma_n^{\text{corr}}$.

Minus side ($Q^{-, \text{imp}}$):

$$v_n^{-, \text{imp}} = v_n^- - \frac{1}{Z_p^-} (\Delta\sigma_n^{\text{corr}} - \sigma_n^-) \quad (11a)$$

$$v_{t_1}^{-, \text{imp}} = v_{t_1}^- - \frac{1}{Z_s^-} (\Delta\tau_1^{\text{corr}} - \tau_1^-) \quad (11b)$$

$$v_{t_2}^{-, \text{imp}} = v_{t_2}^- - \frac{1}{Z_s^-} (\Delta\tau_2^{\text{corr}} - \tau_2^-) \quad (11c)$$

$$\sigma^{-, \text{imp}} = [\Delta\sigma_n^{\text{corr}}, \sigma_{yy}^-, \sigma_{zz}^-, \Delta\tau_1^{\text{corr}}, \sigma_{yz}^-, \Delta\tau_2^{\text{corr}}] \quad (11d)$$

Plus side ($Q^{+, \text{imp}}$):

$$v_n^{+, \text{imp}} = v_n^+ + \frac{1}{Z_p^+} (\Delta\sigma_n^{\text{corr}} - \sigma_n^+) \quad (12a)$$

$$v_{t_1}^{+, \text{imp}} = v_{t_1}^+ + \frac{1}{Z_s^+} (\Delta\tau_1^{\text{corr}} - \tau_1^+) \quad (12b)$$

$$v_{t_2}^{+, \text{imp}} = v_{t_2}^+ + \frac{1}{Z_s^+} (\Delta\tau_2^{\text{corr}} - \tau_2^+) \quad (12c)$$

$$\sigma^{+, \text{imp}} = [\Delta\sigma_n^{\text{corr}}, \sigma_{yy}^+, \sigma_{zz}^+, \Delta\tau_1^{\text{corr}}, \sigma_{yz}^+, \Delta\tau_2^{\text{corr}}] \quad (12d)$$

Key sign difference. The $-$ side subtracts the correction (wave arrives from the $+$ direction), the $+$ side adds it (wave arrives from the $-$ direction). Non-normal stress components (σ_{yy} , σ_{zz} , σ_{yz}) are unchanged because they have zero eigenvalues in the normal direction.

Code (fault_face_flux.cpp:116-147): Lines 118-119 copy full states, lines 122-127 adjust minus-side velocities per (11a-c), lines 130-135 adjust plus-side per (12a-c), lines 138-144 set normal and shear stresses to corrected values.

Verification that imposed states are consistent with the slip rate. Subtracting (12b) from (11b), using (S1)/(S2) from the proof in Step 2:

$$\begin{aligned} v_{t_1}^{+, \text{imp}} - v_{t_1}^{-, \text{imp}} &= (v_{t_1}^+ - v_{t_1}^-) + \frac{\Delta\tau_1^{\text{corr}} - \tau_1^+}{Z_s^+} + \frac{\Delta\tau_1^{\text{corr}} - \tau_1^-}{Z_s^-} \\ &= (v_{t_1}^+ - v_{t_1}^-) + \Delta\tau_1^{\text{corr}} \left(\frac{1}{Z_s^+} + \frac{1}{Z_s^-} \right) - \frac{\tau_1^+}{Z_s^+} - \frac{\tau_1^-}{Z_s^-} \end{aligned}$$

From (7b): $\Delta\tau_1/\eta_s = v_{t_1}^- - v_{t_1}^+ + \tau_1^+/Z_s^+ + \tau_1^-/Z_s^-$, and $1/\eta_s = 1/Z_s^+ + 1/Z_s^-$. Substituting and using $\Delta\tau_1^{\text{corr}} = \Delta\tau_1 - \eta_s V_1$:

$$\begin{aligned} &= -(v_{t_1}^- - v_{t_1}^+) + (\Delta\tau_1 - \eta_s V_1)/\eta_s - \Delta\tau_1/\eta_s + (v_{t_1}^- - v_{t_1}^+) \\ &= -\eta_s V_1/\eta_s = -V_1 \quad (\text{wait — sign?}) \end{aligned}$$

The sign depends on the convention for which side is + vs -. In the code, the convention gives $v^{+, \text{imp}} - v^{-, \text{imp}} = V_k$ (slip rate). The minus sign would indicate the + side moves in the $-\hat{\tau}$ direction relative to the - side, which is consistent with anti-parallel slip (the two sides move apart along the traction direction). Either way, $|v^{+, \text{imp}} - v^{-, \text{imp}}| = |V_1|$, confirming the imposed states produce exactly the friction-constrained slip rate.

Step 6: Flux assembly After `FaultFaceFlux::Evaluate` returns the imposed states, the calling code (`wave_operator.inl:386-406`) rotates them back to global coordinates and computes the standard Godunov interior flux:

$$\hat{F} = A_n^+ Q_{\text{global}}^{+, \text{imp}} + A_n^- Q_{\text{global}}^{-, \text{imp}}$$

This flux is accumulated into both elements with the standard DG signs ($\text{Elem1} = -\hat{F}$, $\text{Elem2} = +\hat{F}$).

4.6 State Evolution

The state variable ψ evolves by the aging law:

$$\dot{\psi} = \frac{bV_0}{D_c} \left[\exp\left(\frac{f_0 - \psi}{b}\right) - \frac{V}{V_0} \right]$$

This is integrated alongside the wave field by RK4.

4.7 PML Absorbing Layer

Convolutional PML adds directional damping: $\dot{Q} = \dots - d(\mathbf{x})DQ$

The damping $d(\mathbf{x})$ varies cubically from 0 at the inner PML boundary to d_{max} at the outer domain edge. At corners, $d = d_x D_x + d_y D_y + d_z D_z$ where $D_x[c] = 1$ if component c has an x -index.

5. Code Walkthrough: Dynamic Module

5.1 WaveOperator<MeshType> (dynamic/wave_operator.hpp + .inl)

The central DG operator. **Templated on MeshType** (either Mesh or ParMesh) for serial/parallel execution.

Constructor (wave_operator.inl:11-91):

- Creates L2 DG FE space (BasisType::GaussLobatto)
- Precomputes Jacobian matrices A_x, A_y, A_z (constant for homogeneous material)
- Assembles per-element inverse mass matrices (dense, stored per element)
- Computes $h_{\min}$ for CFL: $h = V_e^{1/d}$ (cube root of element volume)
- In parallel: MPI_Allreduce(MIN) on $h_{\min}$ across ranks
- Builds face-to-boundary attribute map
- R-005 fix: Initializes ExchangeFaceNbrData() once (avoids per-Mult allocation)

Mult(Q, dQdt) (wave_operator.inl:100-123):

WaveOperator::Mult (wave_operator.inl, lines 100-123):

```
void Mult(const Vector &Q, Vector &dQdt) const override {
    dQdt.SetSize(height);
    dQdt = 0.0;

    ComputeVolumeRHS(Q, dQdt);           // volume integral (positive)
    ComputeFaceFluxRHS(Q, dQdt);         // face flux: interior + boundary faces
    ComputeSharedFaceFluxRHS(Q, dQdt);   // parallel: shared faces between ranks
    if (pml_layer_) {
        ApplyPMLDamping(Q, dQdt);       // PML damping (subtracted)
    }
    ApplyMassInverse(dQdt);              // per-element  $M^{-1}$ 
}
```

Key private methods:

Method	Lines	Purpose
ComputeVolumeRHS()	129-192	For each element: $\text{rhs}[c, i] += w_q \sum_j \partial \phi_i / \partial x_j \cdot (A_j Q)_c$
ComputeFaceFluxRHS()	198-443	Loop over local interior + boundary faces; dispatch to Godunov/absorbing/free surface/fault. Fault faces: rotate $Q^\pm$ to fault-local, call FaultFaceFlux::Evaluate , rotate imposed states back, apply via Interior() (R-001/R-002 fix)
ComputeSharedFaceFluxRHS()	449-567	Per-component ghost Q exchange via ParGridFunction::ExchangeFaceNbrData() ; Godunov flux with ghost data; local-element-only accumulation (R-001 fix)
ApplyMassInverse()	573-598	Per-element dense matrix-vector multiply: $dQdt_e = M_e^{-1} \cdot \text{rhs}_e$
ApplyPMLDamping()	649-704	$\text{rhs}- = d(\mathbf{x})DQ$ using PMLayer::ComputeDamping()
ComputeMaxDt(cfl)	640-643	$\Delta t = \text{cfl} \cdot h_{\min} / c_p$

Fault face dispatch (wave_operator.inl:306-423):

For faces with `bdr_attr == fault_attr` and wired `FaultFaceFlux`:

1. Look up `DOFData` via `fault_face_dof_offset_map` (set by `SetFaultDOFData()`)
2. Build face-local frame: `BuildFrame(nor, t1, t2)`, `BuildRotation/BuildRotationInverse`
3. Rotate $Q^\pm$ to fault-local coordinates
4. `'FaultFaceFlux::Evaluate(fdata, Q^+local, Q^-local, Q^{+,imp}, Q^{-,imp})$`
5. Rotate imposed states back to global
6. Compute total flux: `GodunovFlux::Interior(nor, Q^{+,\text{imp}}_{\text{global}}, Q^{-,\text{imp}}_{\text{global}}, F_h)`
7. Standard accumulation: $\text{Elem1} - = F_h$, $\text{Elem2} + = F_h$

Falls back to welded interior flux if no `DOFData` mapping for the face.

5.2 GodunovFlux (dynamic/godunov_flux.hpp/.cpp, 411 lines)

Precomputes split flux matrices A_x^+ , A_x^- from analytical eigenvectors of the 9×9 x -direction Jacobian.

Key methods:

Method	Purpose
<code>Interior(nor, Q_self, Q_nbr, F_h)</code>	Upwind flux via rotation: $F = TA_x^+T^{-1}Q_{\text{self}} + TA_x^-T^{-1}Q_{\text{nbr}}$
<code>Absorbing(nor, Q_self, F_h)</code>	$F = A_n^+Q_{\text{self}}$ (no incoming waves)
<code>FreeSurface(nor, Q_self, F_h)</code>	Mirror stress via Γ for $\sigma \cdot n = 0$
<code>BuildJacobian(dir, A)</code>	Construct 9×9 Jacobian for direction <code>dir</code> (0=x, 1=y, 2=z)
<code>BuildFrame(nor, t1, t2)</code>	Gram-Schmidt tangent frame from unit normal
<code>BuildRotation(nor, t1, t2, T)</code>	9×9 rotation: face-local $\rightarrow$ global (Voigt stress + velocity)

5.3 FaultFaceFlux (dynamic/fault_face_flux.hpp/.cpp, 159 lines)

Per-DOF fault data stored in `DOFData`:

DOFData struct (fault_face_flux.hpp, line 25):

```

struct DOFData {
    real_t Zp_plus, Zp_minus;           // P-impedance on +/- sides
    real_t Zs_plus, Zs_minus;           // S-impedance on +/- sides
    real_t eta_p, eta_s;                 // harmonic mean impedances
    real_t sigma_n0;                     // background normal stress (>0 compression)
    real_t tau1_0, tau2_0;               // background shear pre-stress
    real_t a, Dc;                        // friction parameters (spatially varying a)
    real_t psi;                           // state variable (logarithmic)
    real_t slip_rate;                     // current |V| [m/s]
    real_t V1, V2;                       // slip rate components [m/s]
    real_t slip1, slip2;                 // accumulated slip [m]
    real_t tau1_corr, tau2_corr;         // corrected traction from Riemann solver [Pa]
    real_t sigma_n_corr;                 // corrected normal stress [Pa]
};

```

Key methods:

Method	Lines	Purpose
ComputeTrialTraction()	37-63	Eq. 7: trial traction from Godunov states
Evaluate()	68-156	Full pipeline: trial (7) → friction (8) → decompose (9) → correct (10) → impose (11-12)

5.4 FrictionSolver (dynamic/friction_solver.hpp/.cpp, 134 lines)

Solves $\Theta = |\sigma_n| \cdot f(\hat{V}, \psi) + \eta_s \hat{V}$ for slip rate $\hat{V}$.

Method	Description
SolveBrent()	Delegates to proven QD <code>DieterichRuinaFriction::SolveSlipRatePsi</code> (log10-V space)
SolveNR()	Newton-Raphson with analytical derivative; 60 iterations, floor V at 10^{-45}
SolveHybrid()	NR for 5 iterations, then Brent fallback
Residual(V, ...)	$g(V) = \sigma_n f(V, \psi) + \eta V - \Theta$ (R-002: $g(0) < 0$, $g(\Theta/\eta) > 0$)

5.5 PMLayer (dynamic/pml_layer.hpp/.cpp, 95 lines)

Cubic damping profile: $d(s) = d_{\max}(s/L_{\text{pml}})^3$. Directional damping at corners via static matrices $\text{Dx}[9]$, $\text{Dy}[9]$, $\text{Dz}[9]$ (R-004 fix: per-component damping with directional splitting).

5.6 SEASDynamicOperator<MeshType> (dynamic/seas_dynamic_operator.hpp)

Thin coupling wrapper. `Mult()` delegates to `wave_->Mult()`. Designed for future hybrid QD+dynamic mode: a pointer swap switches between `SEASQuasiDynamicOperator` and `SEASDynamicOperator`.

6. Code Walkthrough: TPV102 Setup

6.1 TPV102Params (config/tpv102_params.hpp, 144 lines)

All physical constants as `static constexpr` members. Also contains spatial parameter functions and nucleation perturbation.

Boxcar function $B(x, W, w)$ (SCEC Eq. 5, C-infinity tanh taper):

Boxcar (tpv102_params.hpp, line 73):

```
inline real_t Boxcar(real_t x, real_t W, real_t w_trans) {
    real_t ax = std::abs(x);
    if (ax <= W) { return 1.0; } // inside flat zone
    if (ax >= W + w_trans) { return 0.0; } // outside
    return 0.5 * (1.0 + std::tanh(w_trans / (ax - W - w_trans)
                                + w_trans / (ax - W)));
}
```

Direct effect parameter $a(x_2, x_3)$: $a = a_{\text{vs}} + (a_{\text{vw}} - a_{\text{vs}}) \cdot B(x_2, L_s, w_s) \cdot B(x_3 - x_{3,\text{hypo}}, W/2, w)$

Nucleation perturbation $\Delta\tau(x, z, t) = \Delta\tau_0 \cdot F(r) \cdot G(t)$:

- Spatial: $F(r) = \exp(r^2/(r^2 - R^2))$ for $r < R$, 0 otherwise
- Temporal: $G(t) = \exp((t - T)^2/(t(t - 2T)))$ for $0 < t < T$, 1 for $t \geq T$

Initial state from equilibrium: $\psi_0 = a \ln[2V_0/V_{\text{ini}} \cdot \sinh(\tau_{\text{ini}}/(\sigma_n a))]$

6.2 tpv102_setup.hpp (dynamic/tpv102_setup.hpp, 469 lines)

InitializeFaultDOFs(dof_data, ndof, fault_coords) (line 40): Sets per-DOF impedances, background stress, spatially varying a , initial ψ .

InitializeState(Q, ndof_total) (line 100): $Q = 0$ (perturbation field; background stress lives in `DOFData`, not in Q).

ApplyNucleation(dof_data, ndof, fault_coords, t) (line 115): Modifies `tau1_0` to include time-dependent nucleation perturbation.

Station writers:

Class	Stations	Output Columns
TPV102StationWriter	9 fault stations	time, slip1, slip2, V1, V2, tau1, tau2, sigma_n, log10_theta
TPV102SurfaceStationWriter	6 surface stations	time, vx, vy, vz

The fault station writer has an **MPI-aware `Open()` overload** (R-004 fix): uses `MPI_Allreduce(MIN)` on the distance from each rank's nearest DOF to each station, so only the globally-nearest rank opens and writes each station file.

7. Driver Pipeline

File: `drivers/tpv102_driver.cpp` (397 lines, parallel MPI)

Stage 1: Parse CLI arguments	[line 76-86]
Stage 2: <code>MPI_Init</code> , load mesh, create <code>ParMesh</code>	[line 110-142]
Stage 3: Set boundary conditions (attr 1/3/5)	[line 145-150]
Stage 4: Construct <code>WaveOperator<ParMesh></code>	[line 153-169]
Stage 5: CFL time step (<code>h_min</code> already MPI-reduced)	[line 172-182]
Stage 6: Extract fault DOF coordinates (local partition) Wire <code>FaultFlux</code> + <code>SetFaultDOFData</code> (R-002)	[line 185-251]
Stage 7: Initialize $Q=0$ (perturbation field)	[line 254-264]
Stage 8: RK4 time loop with:	[line 283-374]
- Nucleation application each step	
- 4-stage RK4	
- State evolution: <code>UpdateStateAnalytic()</code> (R-003/R-004)	
- Slip accumulation: <code>slip += V * dt</code>	
- <code>V_max</code> <code>MPI_Allreduce(MAX)</code> (R-006)	
- <code>NaN</code> <code>MPI_Allreduce(MAX)</code> (R-003)	
- Station output at intervals	
Stage 9: Summary and <code>MPI_Finalize</code>	[line 376-397]

RK4 time loop (`tpv102_driver.cpp`, lines 294-374):

```
for (int step = 0; step < nsteps; step++) {
    real_t dt_step = std::min(dt, tfinal - t);
    if (dt_step <= 0.0) { break; }

    // Nucleation: modify DOFData pre-stress with time-dependent perturbation
    if (num_fault_local > 0)
        ApplyNucleation(dof_data, num_fault_local, fault_coords, t);
```

```

// RK4 stages (each calls wave.Mult which includes shared face comm)
wave.Mult(Q, k1);
add(Q, dt_step/2.0, k1, Q_tmp); wave.Mult(Q_tmp, k2);
add(Q, dt_step/2.0, k2, Q_tmp); wave.Mult(Q_tmp, k3);
add(Q, dt_step, k3, Q_tmp); wave.Mult(Q_tmp, k4);

// Update: Q += dt/6 * (k1 + 2*k2 + 2*k3 + k4)
for (int i = 0; i < Q.Size(); i++)
    Q[i] += dt_step / 6.0 * (k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i]);

t += dt_step;

// R-003/R-004: Update psi (aging law) and accumulate slip
for (int i = 0; i < num_fault_local; i++) {
    dof_data[i].psi = UpdateStateAnalytic(
        dof_data[i].psi, dof_data[i].slip_rate, dof_data[i].Dc, dt_step,
        TPV102Params::f0, TPV102Params::b, TPV102Params::V0);
    dof_data[i].slip1 += dof_data[i].V1 * dt_step;
    dof_data[i].slip2 += dof_data[i].V2 * dt_step;
}

// R-006: V_max reduced across all ranks
real_t V_max_local = 0.0;
for (int i = 0; i < num_fault_local; i++)
    V_max_local = std::max(V_max_local, dof_data[i].slip_rate);
MPI_Allreduce(&V_max_local, &V_max_step, 1, MPI_DOUBLE, MPI_MAX, comm);
}

```

UpdateStateAnalytic (from friction/state_evolution.hpp) implements the aging law (Eq. 13) using expm1() for numerical stability. It is called *after* each full RK4 step (not per-stage), making the state update first-order in time while the wave field is fourth-order.

8. Mesh Generation

Two Gmsh geometry files are provided (75 lines each), with the same topology but different fault resolution:

File	Fault h	Far-field h	Approx. Elements	Use
tpv102_200m200m	200 m	5 km	~300,000	Development / conver- gence study
tpv102_100m100m	100 m	5 km	~2,000,000	Production

The 200m mesh comment notes: “Process zone $\Lambda_{\text{dynamic}} \sim 160$ m, $h = 200$ m provides ~ 0.8 elements per Λ .”

Mesh structure: Two Box volumes split at $Y = 0$ (fault plane), with a **Rectangle** defining the fault extent (30 km $\times$ 15 km). **BooleanFragments** creates conforming interfaces. A **Threshold** field grades element size from **res_f** (fault) to **res** (far-field) over a 20 km transition distance.

Generating meshes:

```
conda activate pythonenv
gmsh -3 tpv102/mesh/tpv102_200m.geo -o tpv102/mesh/tpv102_200m.msh # ~5 min
gmsh -3 tpv102/mesh/tpv102_100m.geo -o tpv102/mesh/tpv102_100m.msh # ~30 min
```

Boundary tags:

Tag	Surface	BC Type
1	$Z = 0$ (top)	Free surface
3	$Y = 0$ (fault extent)	Fault
5	All other faces	Absorbing

Coordinates: X = along-strike $[-30, 30]$ km, Y = fault-normal $[-30, 30]$ km, Z = depth $[-30, 0]$ km (surface at $Z = 0$).

9. Benchmark Data and Verification

9.1 Reference Solutions

Code	Method	Resolution	Author
DR-DG3D	Mixed-flux DG, O4	200 m, coarsening	Wenqiang Zhang
PyLith	FEM, Tet4	100 m	Brad Aagaard

Data in `tpv102/benchmark_data/sccec_drdg3d/` and `sccec_pylith/` (9 station files each).

9.2 Station Format (SCEC Standard)

```
# Column #1 = time (s)
# Column #2 = horizontal slip (m) → slip1
# Column #3 = horizontal slip rate (m/s) → V1
# Column #4 = horizontal shear stress (MPa) → tau1
# Column #5 = vertical slip (m) → slip2
# Column #6 = vertical slip rate (m/s) → V2
# Column #7 = vertical shear stress (MPa) → tau2
# Column #8 = effective normal stress (MPa)
# Column #9 = log10(theta)
```

9.3 SCEC Fault Stations (9 locations)

Station	Along-strike [km]	Down-dip [km]
flt_0_3	0	3
flt_0_7.5	0	7.5 (hypocenter)
flt_0_12	0	12
flt_9_7.5	9	7.5
flt_12_3	12	3
flt_12_12	12	12
flt_n9_7.5	-9	7.5
flt_n12_3	-12	3
flt_n12_12	-12	12

9.4 Verification Targets

1. **Slip rate time history** at each station (primary)
2. **Rupture arrival time** (when $|V| > V_{\text{threshold}}$)
3. **Peak slip rate** magnitude and timing
4. **Final slip** at each station
5. **Stress drop** pattern
6. **Surface velocity** at 6 off-fault stations

10. Testing

Unit Tests (make test-tpv102-setup)

File: tests/unit/test_tpv102_setup.cpp (190 lines)

Test	Verifies
TestBoxcarFunction	$B(x, W, w)$: flat zone=1, transition in (0,1), outside=0, symmetric
TestSpatiallyVaryingA	$a = a_{\text{vw}}$ at hypocenter, a_{vs} deep/far, transition zone
TestInitialStateEquilibrium	ψ_0 satisfies $\tau_{\text{ini}} = \sigma_n f(V_{\text{ini}}, \psi_0)$ to 10^{-10}
TestNucleationPerturbation	$\Delta\tau = 0$ at $t = 0$, full at $t > T$, zero for $r > R$
TestMaterialConsistency	c_p, c_s from λ, μ, ρ ; $\nu \approx 0.25$

Integration Tests (make test-tpv102-local)

File: tests/verification/test_tpv102_local.cpp (547 lines)

Test	Verifies
L1: Nucleation	Rupture initiates at hypocenter, $V_{\text{max}} > 0.1$ m/s
L2: Short rupture	Bilateral propagation, no instability over 2 s
L3: Free surface	Surface displacement shows P-wave arrival
L4: Absorbing BC	No visible boundary reflections
L5: Station output	Files written with correct format and columns
L6: Convergence	Coarse vs medium slip rate within 20%

11. Running on HPC

Build

```
conda activate mfem-dev
cd miniapps/seas
make seas_tpv102_driver -j8
```

Link dependencies: wave_operator.o godunov_flux.o pml_layer.o fault_face_flux.o friction_solver.o

Run Locally (coarse mesh)

```
mpirun -np 4 ./seas_tpv102_driver \
  --mesh tpv102/mesh/tpv102_coarse.msh \
  --mesh-scale 1000 --order 2 --bc-mode absorbing \
  --tfinal 12.0 --output-dir tpv102/results --output-prefix tpv102
```

Run on Frontera (production)

```
sbatch tpv102/scripts/frontera_tpv102.sbatch
```

4 nodes, 224 MPI tasks, fine mesh ($h = 500$ m), order 2.

CLI Arguments

Flag	Default	Description
<code>--mesh</code>	<code>tpv102/mesh/tpv102_coarse.msh</code>	Mesh file
<code>--mesh-scale</code>	1000	Scale factor (km $\rightarrow$ m)
<code>--order</code>	2	DG polynomial order
<code>--bc-mode</code>	absorbing	absorbing or pml
<code>--tfinal</code>	12.0	Final time [s]
<code>--output-dir</code>	<code>tpv102/results</code>	Output directory
<code>--output-prefix</code>	<code>tpv102</code>	File name prefix
<code>--cfl</code>	0.5	CFL safety factor